

Implement and Benchmark Advanced Code Splitting Strategies in a React Application

Back

Mandatory

Domain

Frontend Development

Difficulty

Advanced

Industries

E-commerce

SaaS

Skills

Frontend Development

Performance Optimization

Solution Architecture

Technical Analysis

UX Analysis

Asynchronous Loading

Build Configuration

Tools

Chart.js

Docker

Lighthouse

Node.js

React

TipTap

Vite



Pending Submission

Please submit your work when ready.

Deadline: 16 May 2026, 04:59 pm

TIME REMAINING

4d

Overview

Instructions

Resources

Submit

Report Issue

Build a multi-page React application to implement and benchmark four distinct code-splitting strategies: none, route-based, component-based, and library-based. You will learn to use `React.lazy`, `Suspense`, and Vite's `manualChunks` to optimize application performance. This task is crucial for developing production-grade web applications that deliver fast initial load times and efficient resource caching.

Description

1. Project Overview & Core Concepts

Modern web applications have grown increasingly complex, often leading to large JavaScript bundles that can significantly slow down initial page loads. A slow start can frustrate users and negatively impact key business metrics. The primary technique to combat this is **code splitting**, a process of dividing the application's code into smaller, more manageable chunks that can be loaded on demand.

In this project, you will build a React application from the ground up and systematically apply and benchmark four different code-splitting strategies. Your goal is not just to implement these techniques but to deeply understand their impact on performance, caching, and user experience. You will use industry-standard tools to measure bundle size, analyze network waterfalls, and quantify improvements in key metrics like Time to Interactive (TTI).

Key Technologies and Concepts

- 1. React (`React.lazy` and `Suspense`):** React provides built-in tools for code splitting. `React.lazy` lets you render a dynamically imported component as a regular component. It works seamlessly with `Suspense`, which allows you to specify a loading indicator (a fallback UI) while the lazy component's code is being loaded over the network.
 - 1. Why it matters:** This is the idiomatic way to implement route-based and component-based splitting in React. It integrates directly with the React component model, making it declarative and easy to reason about. A common pitfall is forgetting that `React.lazy` currently only supports default exports.
- 2. Dynamic `import()`:** This is the underlying JavaScript language feature that makes code splitting possible. Unlike a static `import` statement, which bundles the module at build time, a dynamic `import()` is a function-like expression that returns a Promise. The module is fetched only when the `import()` is called, allowing for on-demand loading.

1. **Why it matters:** It's the engine behind `React.lazy`. Understanding how it works will help you debug issues and even

partnr

Dashboard Skill Graph Profile GPP

the build output. The `build.rollupOptions.output.manualChunks` option lets you define rules to group specific modules into custom chunks. This is particularly useful for separating large, stable third-party libraries (vendor code) from your more frequently changing application code.

1. **Why it matters:** This strategy dramatically improves caching. When you deploy a small change to your app, users only need to re-download the small app code chunk, not the entire vendor library, which their browser has likely cached from a previous visit.
4. **Performance Benchmarking (Lighthouse & Chrome DevTools):** You can't optimize what you can't measure. Lighthouse is an automated tool for auditing web page quality, providing scores for performance, accessibility, and more. The Chrome DevTools Network tab provides a waterfall view of all resources loaded by a page, showing what is being loaded, when, and from where (network or cache).
 1. **Why it matters:** These tools provide the objective data needed to prove that your optimizations are working. They help you move from "I think this is faster" to "I have reduced the initial JS payload by 150KB and improved TTI by 400ms."

2. System Architecture & Setup

You will build a simulated SaaS application with four distinct pages, each with different dependency weights to make the effects of code splitting clearly visible.

1. `/dashboard` : A lightweight landing page.
2. `/analytics` : A data-heavy page that uses `chart.js` for visualization.
3. `/editor` : A feature-rich page with a `TipTap`-based rich text editor.
4. `/settings` : Another lightweight page for user settings.

The core of this project involves creating four separate build configurations to compare their outputs. You should set up your `package.json` with distinct scripts for each version.

```
{
  "scripts": {
    "dev": "vite",
    "build:nosplit": "vite build --config vite.config.nosplit.js",
    "build:routes": "vite build --config vite.config.routes.js",
```

```
"build:components": "vite build --config vite.config.components.js",
```

partnr

[Dashboard](#)[Skill Graph](#)[Profile](#)[GPP](#)

```
}
```

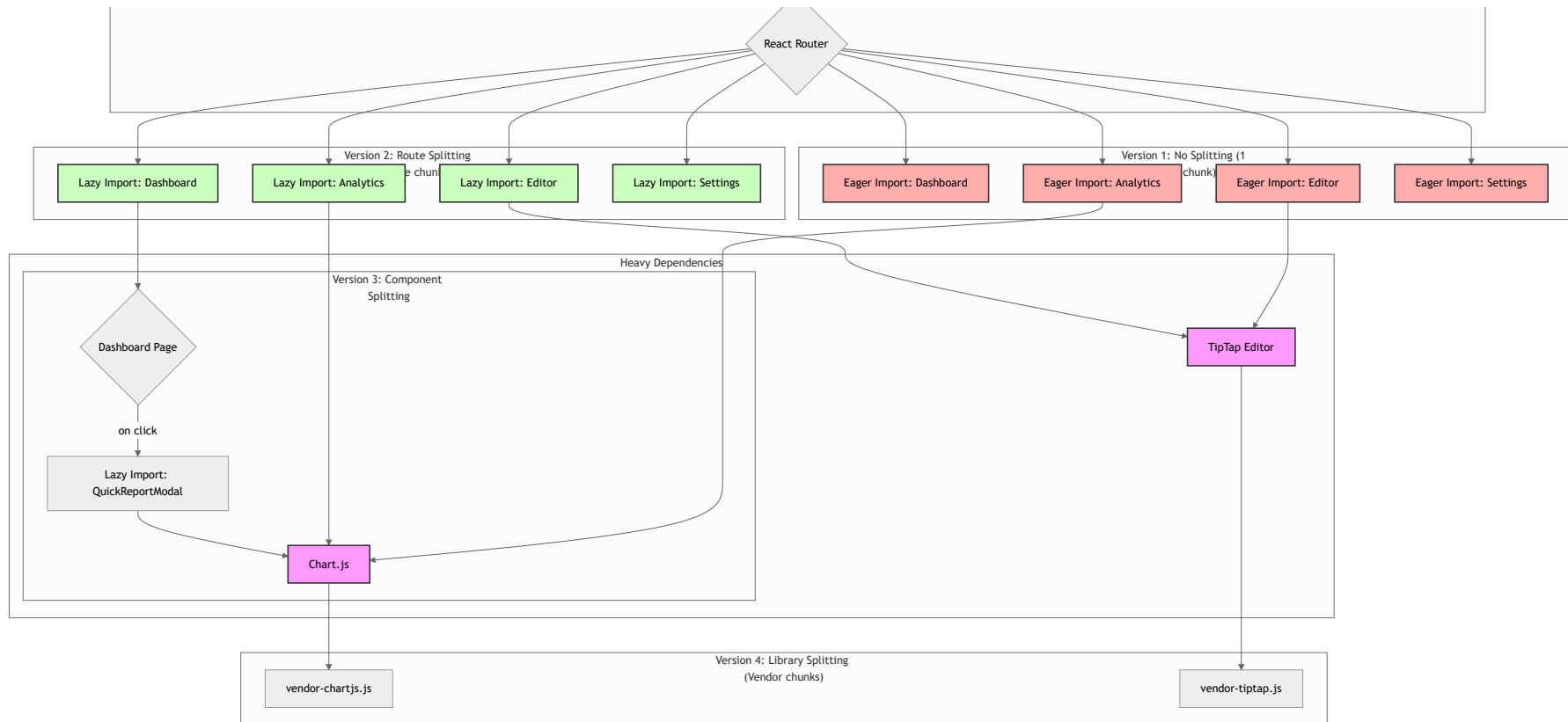
This structure allows you to generate each version's build artifacts into a separate directory for isolated analysis. The application itself will use React Router v6 for navigation.

Architectural Diagram

The following diagram illustrates the different loading strategies you will implement:

partnr

Dashboard Skill Graph Profile GPP



As the diagram shows, you'll progress from a monolithic bundle to increasingly granular and intelligent chunking strategies, ultimately separating application code from third-party libraries for maximum cache efficiency.

3. Implementation Step-by-Step: Building the Four Versions

Follow these steps to create each version of the application.

Version 1: The Monolith (Baseline)

This version serves as your performance baseline. All code is bundled into a single file.

1. **Project Setup:** Initialize a new React project using Vite.

partnr

Dashboard Skill Graph Profile GPP

3. **Create Pages:** Build the four page components (`Dashboard` , `Analytics` , `Editor` , `Settings`). Ensure the `Analytics` page renders a `Chart.js` chart and the `Editor` page renders a `TipTap` instance.
4. **Set Up Routing:** In your main `App.jsx` , use static imports for all page components and configure the routes using React Router.

```
// App.jsx - Version 1
import Dashboard from './pages/Dashboard';
import Analytics from './pages/Analytics';
import Editor from './pages/Editor';
import Settings from './pages/Settings';

// ... Router setup with <Routes> and <Route> ...
```

5. **Build:** Run your `build:nosplit` script. Inspect the output directory (`dist`). You should see one large JavaScript file containing all your application code and dependencies.

Version 2: Route-Based Splitting

This is the most common and impactful form of code splitting.

1. **Refactor Imports:** Modify `App.jsx` to use `React.lazy` and dynamic `import()` for each page component.

```
// App.jsx - Version 2
import { lazy, Suspense } from 'react';

const Dashboard = lazy(() => import('./pages/Dashboard'));
const Analytics = lazy(() => import('./pages/Analytics'));
// ... etc.
```

2. **Add Suspense:** Wrap your `<Routes>` component with a `<Suspense>` boundary. Provide a fallback UI, such as a simple loading spinner or a page skeleton, to show while the route's chunk is loading.

partnr

Dashboard Skill Graph Profile GPP

```

    </Routes>
  </Suspense>

```

3. **Build:** Run your `build:routes` script. The output directory should now contain multiple smaller JavaScript chunks, one for each lazily loaded page, in addition to the main entry chunk.

Version 3: Component-Based Splitting

Here, you'll apply splitting to a heavy component *within* a page.

1. **Create a Heavy Component:** On your `Dashboard` page, create a `QuickReportModal` component that uses `Chart.js`. This modal will be shown only when a user clicks a button.
2. **Implement Lazy Loading:** In the `Dashboard` component, lazily import the `QuickReportModal`. Use a state variable to control its visibility.

```

// Dashboard.jsx - Version 3
const QuickReportModal = lazy(() => import('../components/QuickReportModal'));

function Dashboard() {
  const [showModal, setShowModal] = useState(false);
  return (
    <div>
      <button onClick={() => setShowModal(true)}>View Quick Report</button>
      {showModal && (
        <Suspense fallback={<div>Loading report...</div>}>
          <QuickReportModal onClose={() => setShowModal(false)} />
        </Suspense>
      )}
    </div>
  );
}

```

3. **Build:** Run your `build:components` script. You should see a new, separate chunk specifically for `QuickReportModal`.

partnr

Dashboard Skill Graph Profile GPP

This final version optimizes for caching by separating vendor code.

1. **Configure Vite:** Create a `vite.config.libsplit.js` file. Use the `manualChunks` option to define rules for splitting out large libraries.

```
// vite.config.libsplit.js
import { defineConfig } from 'vite';

export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: (id) => {
          if (id.includes('node_modules/chart.js')) {
            return 'vendor-chartjs';
          }
          if (id.includes('node_modules/@tiptap')) {
            return 'vendor-tiptap';
          }
          if (id.includes('node_modules/react')) {
            return 'vendor-react';
          }
        }
      }
    }
  }
});
```

2. **Build:** Run the `build:libsplit` script. Inspect the build output. You will now see chunks named `vendor-chartjs.js`, `vendor-tiptap.js`, and `vendor-react.js`, completely separate from your application code chunks.

4. Benchmarking and Analysis Protocol

With all four versions built, you will now systematically measure and compare their performance. Document your findings for each

partnr

Dashboard Skill Graph Profile GPP

1. For each of the four build outputs, calculate the total gzipped size of all JavaScript assets.
2. More importantly, determine the size of the JavaScript downloaded on an **initial visit to the Dashboard (/) page**. This is the critical metric for first-time user experience. Use `ls -lh` or a similar command to inspect file sizes.

2. Time to Interactive (TTI) Measurement:

1. Use a local server like `npx serve` to host each of the four `dist` directories on different ports.
2. Use the Lighthouse CLI to programmatically measure the TTI for the Dashboard page of each version. Run the audit in a clean browser state (incognito mode) to simulate a first-time visit.
3. Example command:

```
lighthouse http://localhost:4001 --output=json --only-categories=performance --chrome-flags="--headless"
```

4. Parse the JSON output to extract the `interactive` audit's numeric value.

3. Network Waterfall Analysis:

1. Open Chrome DevTools and go to the Network tab. Disable the cache.
2. For each of the four running versions:
 1. Load the Dashboard page (/) and take a screenshot of the JS files loaded.
 2. Navigate to the Analytics page (/analytics) and take a screenshot of the *additional* JS files loaded.
 3. Navigate to the Editor page (/editor) and take a screenshot of the *additional* JS files loaded.
3. Your screenshots should clearly illustrate the on-demand loading behavior of the different strategies.

4. Cache Efficiency Simulation:

1. This step demonstrates the power of library splitting.
2. **Day 1 (Initial Visit):** With a clear cache, visit all four pages of the **Version 4 (Lib Splitting)** application. In the DevTools Network tab, note the total bytes of JavaScript downloaded.

3. **Day 2 (App Update):** Make a trivial change to your `Dashboard.isx` component (e.g., change some text). Re-run the

partnr

Dashboard Skill Graph Profile GPP

etc.) are served from the browser cache (indicated by a 304 Not Modified status or (disk cache) size).

4. Calculate the bytes downloaded on Day 2 and compare it to Day 1. Do the same for the **Version 1 (No Splitting)** build to highlight the difference.

5. **Suspense and `useTransition` UX Evaluation:**

1. Focus on **Version 3 (Component Splitting)**. In DevTools, throttle your network speed to "Slow 3G".
2. Click the "View Quick Report" button. Observe the user experience. The fallback UI appears, but the main UI might feel unresponsive.
3. Now, refactor your button's `onClick` handler to use React's `useTransition` hook.

```
import { useTransition } from 'react';

const [isPending, startTransition] = useTransition();

function handleOpen() {
  startTransition(() => {
    setShowModal(true);
  });
}
```

4. Use the `isPending` state to show a loading indicator *on the button itself* (e.g., disable it and show a spinner). Repeat the test on a throttled network. Document the perceived improvement in UI responsiveness. The page no longer feels stuck while the component chunk is loading.

5. Final Deliverables and Reporting

Your submission must be a comprehensive analysis of your findings.

1. **Source Code:** All four versions of the application, runnable via the `package.json` scripts.

2. Analysis Report (`results/report.md`): A markdown file containing:

partnr

Dashboard Skill Graph Profile GPP

Metric	No splitting	Route splitting	Component splitting	Lib splitting
Initial JS Load (Dashboard) (KB)				
Total JS Size (All Routes) (KB)				
Dashboard TTI (ms)				
Chunks on Initial Load (Count)				
JS from Cache on App Update (%)				

* **Network Screenshots:** Embed the network waterfall screenshots you captured for each key navigation step.

* **Recommendation:** Based on your data, provide a recommended code-splitting strategy for a typical, complex SaaS dashboard applicatio

6. FAQ

1. Q: My `React.lazy()` import is failing with an error about a component not being a valid React component. What's wrong?

1. A: This almost always happens because `React.lazy` requires the imported file to have a `default` export. If your component is a named export (e.g., `export function MyComponent`), it will fail. You can either change it to `export default function MyComponent` or use an adapter in the import statement: `const MyComponent = lazy(() => import('./MyComponent').then(module => ({ default: module.MyComponent })));`. The former is generally cleaner. See the [React documentation](#) for more details.

2. Q: Why do I need four separate Vite config files?

1. A: While you could use environment variables to control a single config, separate files provide maximum clarity and isolation for this exercise. It ensures that the build configuration for one strategy does not accidentally interfere with another, making your benchmarking results clean and reliable.

3. Q: The `manualChunks` configuration seems complex. How do I know what to include in the string?

1. A: The `id` passed to the `manualChunks` function is the absolute file path of the module being processed. The simplest way to create a vendor chunk is to check if the path includes `'node_modules/library-name'`. You can use a bundle analyzer tool

like `vite-bundle-visualizer` to see which libraries are contributing the most to your bundle size, which helps you decide

partnr

Dashboard Skill Graph Profile GPP

Core Requirements

1. The project must be containerized using Docker and Docker Compose for easy evaluation. A `docker-compose.yml` file must be present at the root of the repository.

Contract Specification:

1. **File Location:** `docker-compose.yml` in the project root.
2. **Services:** Must define at least one service (e.g., `app`) to serve the production build of the application (specifically, the output of the `build:libsplitt script`).
3. **Functionality:** Running `docker-compose up` must build the Docker image and start the container, serving the application on a specified port (e.g., 80 or 8080).
4. **Healthcheck:** The application service in `docker-compose.yml` must have a `healthcheck` configured to verify that the web server is running.

Verification:

1. Verify the existence of `docker-compose.yml` and a corresponding `Dockerfile`.
2. Run `docker-compose up -d --build`.
3. Verify the container starts and becomes healthy within 2 minutes.
4. Send an HTTP GET request to the exposed port and expect a 200 OK response with HTML content.

-
2. The `package.json` file must contain four distinct build scripts, one for each code-splitting strategy.

Contract Specification:

1. **File Location:** `package.json`
2. **Required Scripts:** The `scripts` object must contain the following keys:
 1. `build:nosplit`

2. `build:routes`

partnr

Dashboard

Skill Graph

Profile

GPP

Verification:

1. Parse the `package.json` file.
 2. Check for the existence of all four required script names under the `scripts` key.
-

3. The `build:nosplit` script must generate a build with a single primary JavaScript chunk.

Contract Specification:

1. **Trigger:** Run `npm run build:nosplit`.
2. **Output Directory:** A `dist/assets` directory (or similar, based on Vite config).
3. **Expected Chunks:** The output directory should contain exactly one main JavaScript file (e.g., `index-*.js`). Other asset files like CSS, images, and vendor polyfills are acceptable, but the application and its dependencies must be bundled into one primary JS file.

Verification:

1. Execute `npm install` and then `npm run build:nosplit`.
 2. List the contents of the build output's assets directory.
 3. Verify that there is only one JavaScript file matching the pattern for a main application chunk (e.g., `index-*.js` or `main-*.js`).
-

4. The `build:routes` script must generate a build where each page is in a separate JavaScript chunk.

Contract Specification:

1. **Trigger:** Run `npm run build:routes`.
2. **Output Directory:** A `dist/assets` directory.

3. **Expected Chunks:** The output directory must contain multiple JavaScript chunks. There should be chunks corresponding to

1. Execute `npm install` and then `npm run build:routes`.
2. List the contents of the build output's assets directory.
3. Verify the existence of at least 3 distinct JavaScript chunks (e.g., `index-*.js`, `Analytics-*.js`, `Editor-*.js`).

5. The `build:libsplitt` script must generate a build with separate chunks for specified vendor libraries.

Contract Specification:

1. **Trigger:** RUN `npm run build:libsplitt`.
2. **Output Directory:** A `dist/assets` directory.
3. **Expected Chunks:** The output must contain chunks with specific names for the vendor libraries.
 1. A chunk for Chart.js (e.g., `vendor-chartjs-*.js`)
 2. A chunk for TipTap (e.g., `vendor-tiptap-*.js`)
 3. A chunk for React (e.g., `vendor-react-*.js`)

Verification:

1. Execute `npm install` and then `npm run build:libsplitt`.
2. List the contents of the build output's assets directory.
3. Verify that files matching the patterns `vendor-chartjs-*.js`, `vendor-tiptap-*.js`, and `vendor-react-*.js` exist.

6. The application must contain navigation links and key UI elements with `data-testid` attributes for automated testing.

Contract Specification:

1. **Navigation Links:** Must be present for all pages.
 1. Dashboard: ``data-testid=`

7. A detailed analysis report must be provided in a specific markdown file.

partnr

Dashboard

Skill Graph

Profile

GPP

1. **File Location:** `results/report.md`

2. **Required Content:** The file must contain a markdown table with the following headers in the first row:

1. Metric
2. No splitting
3. Route splitting
4. Component splitting
5. Lib splitting

Verification:

1. Check for the existence of the `results/report.md` file.
2. Parse the markdown file.
3. Verify that a table exists and its first row contains the specified headers.

-
8. Screenshots of the browser's network tab must be provided in a designated directory.

Contract Specification:

1. **Directory Location:** `results/screenshots/`

2. **Required Files:** The directory must contain at least four `.png` files, demonstrating the network activity for different loading scenarios.

Verification:

1. Check for the existence of the `results/screenshots/` directory.
2. Verify that it contains at least four files with a `.png` extension.

-
9. The component-based splitting for the 'Quick Report' modal must function correctly.

Contract Specification:

p a r t n r

Dashboard

Skill Graph

Profile

GPP

2. The lazily loaded modal, when rendered, must have a container element with `data-testid="quick-report-modal"` .

2. **Behavior:** The JavaScript chunk for the `QuickReportModal` must only be loaded after the 'View Report' button is clicked.

****Verification (using Playwright/Cypress):**

1. Start the application (using the 'components' version).
2. Navigate to the Dashboard page (/).
3. Start monitoring network requests.
4. Confirm that no chunk corresponding to `QuickReportModal` has been loaded.
5. Click the button with `data-testid="view-report-button"` .
6. Verify that a new network request for the `QuickReportModal` 's JavaScript chunk is initiated.
7. Verify the element with `data-testid="quick-report-modal"` is now visible in the DOM.

10. The `useTransition` hook must be implemented to provide visual feedback while the lazy component is loading.

Contract Specification:**1. UI Elements:**

1. The 'View Report' button (`data-testid="view-report-button"`) should show a loading state.
2. When the transition is pending, the button should contain an element with `data-testid="button-loading-indicator"` .

2. **Behavior:** When the 'View Report' button is clicked on a throttled network, the loading indicator must appear immediately, before the modal's network request completes and the modal itself is rendered.

****Verification (using Playwright/Cypress with network throttling):**

1. Start the application.
2. Enable network throttling (e.g., 'Slow 3G').
3. Navigate to the Dashboard page.

4. Click the button with `data-testid="view-report-button"` .

partnr

Dashboard

Skill Graph

Profile

GPP

[About Us](#) [Contact Us](#) [Privacy Policy](#) [Terms and Conditions](#)

All rights reserved. Copyright, Partnr 2025-26

